
photosmesher

Aleksandar Jovanovic

Dec 30, 2025

CONTENTS

1	Build status	3
2	Quickstart	5
3	Citation	7
4	Acknowledgements	9
4.1	Installation	9
4.2	Usage	9
4.3	Photosmesher API	13
	Python Module Index	15
	Index	17

The *photosmesher* (short for “photographs mesher”) is a tool designed to streamline and automate the process of creating the simulation geometry and mesh generation for plasma modelling. While excellent meshing tools like [gmsht](#) are available, creating an appropriate mesh from scratch remains time-consuming and requires lots of effort. Recognizing that modelling and experimental articles usually include detailed schematic diagrams of simulation domains, electric discharge setups, plasma sources or photographs of discharges, we try to mitigate this effort by converting such photographs into simulation geometry and generating the desired mesh. The generated mesh can be imported into [FEDM](#) or any other solver (tested in [FEniCS](#) and [OpenFOAM](#)).

The *photosmesher* uses the [OpenCV](#) computer vision library to identify elementary shapes in electric discharge photographs or the schematic diagrams. The supplied image is first read and passed to *OpenCV*, where a Canny edge detector and contour-fitting routine isolate the outlines of the structure. These outlines are converted into a *gmsht* geometry via its Python API. The resulting mesh is saved into the *gmsht* native *.msh* format file and converted using [meshio](#) to files suitable to use it in *FEDM* (such as *xml* or *xdmf*). The physical group names chosen by the user are also saved (nothing that their use with *FEDM* and legacy *FEniCS* is limited).

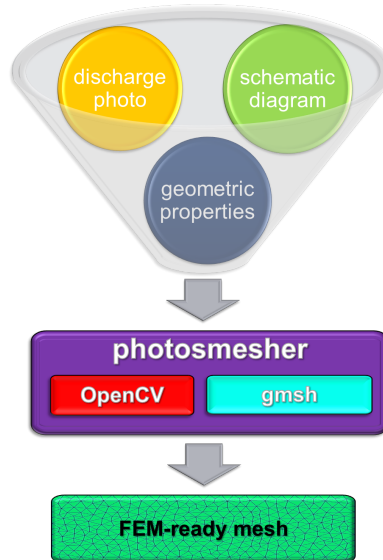


Figure 1: Illustration of the meshing workflow

Additionally to the mesh generation, the tool provides an option for mesh refinement based on intensity in the photos. The general idea is to refine the regions where the intensity is higher, e.g. regions of the streamer propagation and where typically much finer mesh is required to resolve steep gradients.

BUILD STATUS

ver. 0.1

QUICKSTART

The programme is portable, so it can be simply downloaded and used in the following way

```
git clone https://github.com/INP-SDT/modern_techniques_for_optimisation_of_plasma_
↳modelling_codes.git
cd modern_techniques_for_optimisation_of_plasma_modelling_codes/mesh_generation/
↳photosmesher
python3 photosmesher.py
```

Note

It is important to note that the *photosmesher* is still under development. At this stage, it is capable of detecting straight lines within photos; however, users must manually select the lines and points used in building the desired geometry. Future updates will introduce the support for curved boundaries. Additionally, the current version of mesh refinement is based on a *gmsh*'s box field option around high-intensity regions (e.g. streamer photographs).

CITATION

If you use photosmesher for your research, please cite the following work Jovanović A. P. et al. Exploration of modern techniques for optimising plasma modelling procedures DPG-Frühjahrstagung (DPG Spring Meeting) Göttingen 2025

ACKNOWLEDGEMENTS

Supported by DAAD PPP with Serbia - project number: 57703239.

4.1 Installation

The programme is portable and does not need installation. It can be simply executed running following script:

```
python3 photosmesher.py
```

which starts the GUI. Note, however, that prior to its use some additional packages need to be installed:

```
git clone https://github.com/INP-SDT/modern_techniques_for_optimisation_of_plasma_
↳modelling_codes.git
cd modern_techniques_for_optimisation_of_plasma_modelling_codes/mesh_generation/
↳photosmesher
python -m venv env
source env/bin/activate
pip install -r requirements
```

4.2 Usage

The procedure starts by running the programme by executing following command

```
python3 photosmesher.py
```

which opens the main window and starts GUI. The required steps are described below in detail.

Step 1

After starting the GUI, user needs to select the image file (it supports various files, current version was tested with jpg, png, and svg files) by clicking on *choose file* button and select if automated refinement should be used (not that it makes sense to use it with the discharge photo only). The input file needs to contain either the photo of the electric discharge, the geometry sketch or even schematic diagram of the plasma source.

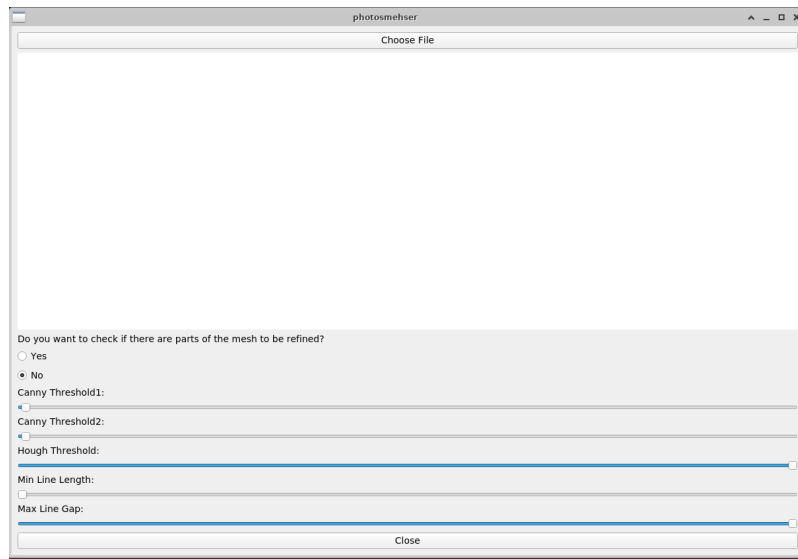


Fig 1. Selecting input file and choosing refinement option

Step 2

After choosing the input file, the photo is loaded and the lines are recognised. Note that the user might need to adjust the parameters using sliders below the canvas to recognise all the lines. The user needs to select all relevant lines and choose *done* button. Each click marks the selected line with the red color. The user can reset selection with middle click or go one selection back with the right mouse click.

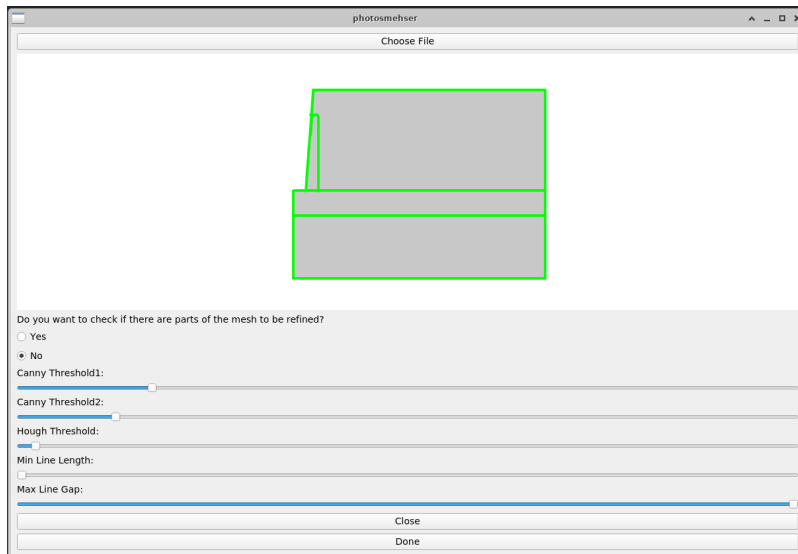


Fig 2. Selecting recognised lines to determine points constructing the geometry

Step 3

The new window shows the intersection points of the selected lines. The user then needs to select the desired points and click on done to proceed to the next step. Each click marks the selected point with the red color. The user can reset selection with middle click or go one selection back with the right mouse click. After choosing all points, user need to click on *Select points* button to proceed with the following step.

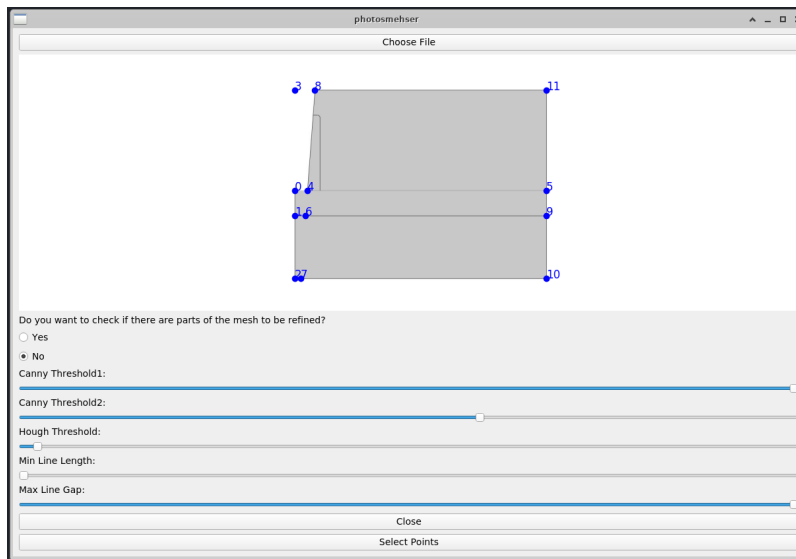


Fig 3. Selecting the desired points constructing the simulation geometry

Step 4

The user now needs to enter the distance between the first two selected points (units are relative, although metre is assumed) in the newly-appeared text box at the bottom of the window and click *Confirm* button.

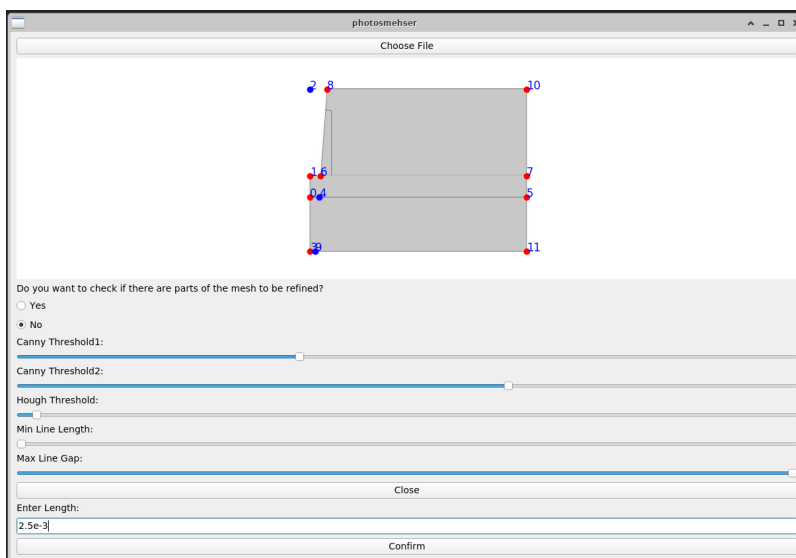


Fig 4. Specifying the reference distance between first two selected points

Attention

Note that the specified distance between the first two selected points determines the length of all the other lines.

Step 5

In the newly opened window, user can modify the values of the coordinates of the chosen points, since the calculated coordinates may defer from the real values due to rounding errors.

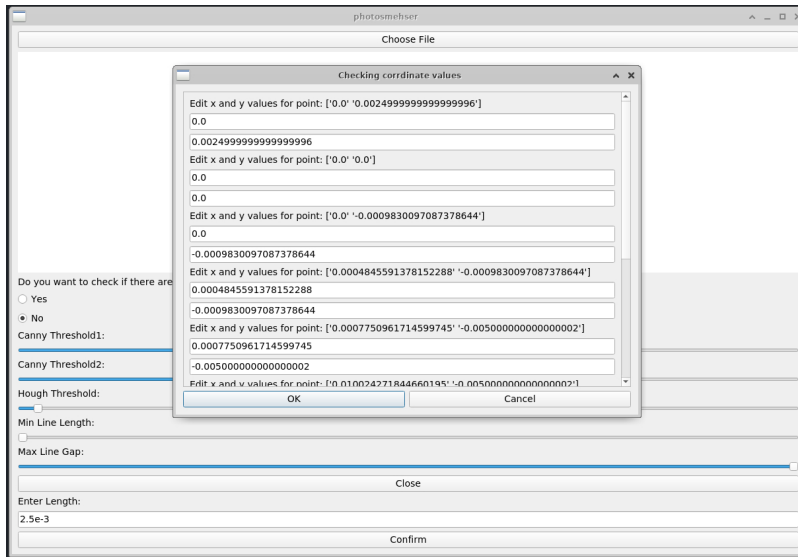


Fig 5. Adjusting the coordinates of selected points

Step 6

The last window asks user to specify the name of the mesh. The mesh will be saved as *gms*h geo and *msh* files, and converted using *meshio* into *xml* and *vtk* files suitable as input for FEDM simulations or plotting in *Paraview*.

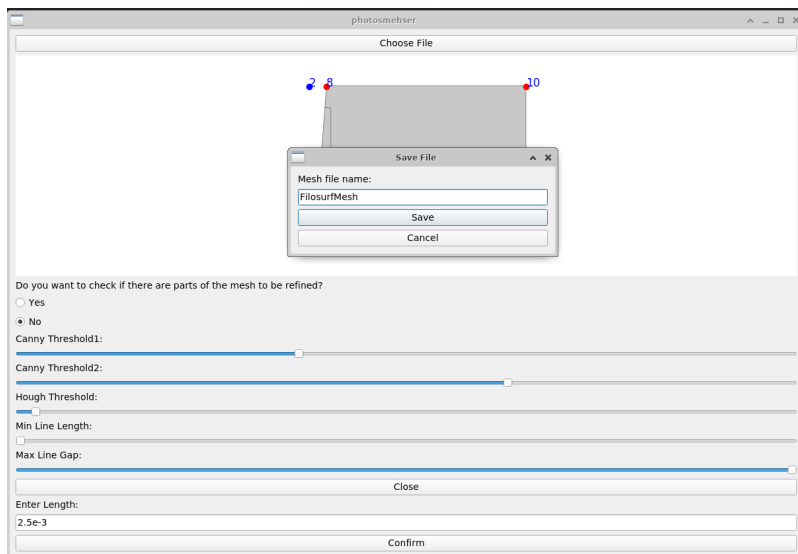


Fig 6. Specifying the output file name

The generated mesh is illustrated below, noting that we did not select any refinement. The prototyped mesh can be refined by loading *geo* file and prescribing refined regions, for example using the *field* function in *gms*h.

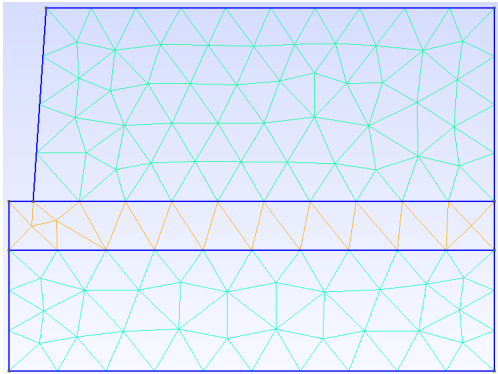


Fig 7. Example of generated (unrefined) mesh using schematic diagram of the plasma source as the input file
In this way, starting from the schematic diagram, we have prototyped the mesh in a few steps in just a few minutes allowing for easy manual adjusting.

4.3 Photosmesher API

<i>photosmesher</i>	Created on Mon Mar 10 12:58:07 2025
---------------------	-------------------------------------

4.3.1 photosmesher

Created on Mon Mar 10 12:58:07 2025
@author: jovanovic with the help of Microsoft Copilot and chatGPT

Classes

App()	Main application window for image processing and mesh generation.
MultiInputDialog(data_array)	Dialog to edit a 2-D array of coordinate pairs.
SaveFileDialog()	Simple dialog to ask the user for a mesh file name.

PYTHON MODULE INDEX

p

photosmesher, [13](#)

INDEX

M

module

 photosmesher, [13](#)

P

photosmesher

 module, [13](#)